

SWE Take-Home Assessment:

Support Ticket System

Time Allowed: 24 hours
Expected Effort: ~2 hours (vibe coding tools encouraged)
Role: Software Engineer (Full-Stack)

Scenario

Build a **Support Ticket System** — a web application where support agents can submit tickets, the system automatically classifies and routes them to the correct team, and managers can track SLA compliance.

The system has two user roles: **Support Agents** (submit and work tickets) and **Team Leads** (view team queues and metrics). Tickets arrive, get classified by type and priority, are routed to the appropriate team, and are tracked through resolution.

Data Model

Ticket

Field	Type	Description
id	UUID	Unique identifier
title	string	Short summary
description	text	Full ticket description
channel	enum	<code>web_form</code> , <code>email</code> , <code>api</code>
status	enum	<code>open</code> → <code>in_progress</code> → <code>resolved</code> → <code>closed</code>
priority	enum	<code>P1</code> (critical), <code>P2</code> (high), <code>P3</code> (medium), <code>P4</code> (low)
category	enum	<code>billing</code> , <code>engineering</code> , <code>account_management</code> , <code>security</code> , <code>general</code>
assigned_team	string (nullable)	Team the ticket is routed to

assigned_agent	string (nullable)	Agent working the ticket
sla_deadline	datetime (nullable)	When this ticket must be assigned by
created_at	datetime	Submission timestamp
updated_at	datetime	Last modified timestamp
resolved_at	datetime (nullable)	When the ticket was resolved

Team

Field	Type	Description
id	UUID	Unique identifier
name	string	Team name
escalation_contact	string	Who to page if SLA is about to breach

Routing Rule

Field	Type	Description
id	UUID	Unique identifier
name	string	Human-readable rule name
condition_field	string	Field to evaluate (<code>category</code> , <code>priority</code> , <code>channel</code>)
condition_operator	string	<code>equals</code> , <code>contains</code> , <code>in</code>
condition_value	string	Match value (JSON-encoded for <code>in</code>)
target_team	string	Team to route to
auto_priority	string (nullable)	Override priority if set (e.g., security → always P1)
priority_order	integer	Evaluation order (lower = first match wins)

Ticket History (append-only)

Field	Type	Description
id	UUID	Unique identifier

ticket_id	UUID	Related ticket
action	string	What happened (<code>created</code> , <code>classified</code> , <code>assigned</code> , <code>status_changed</code> , <code>escalated</code> , <code>resolved</code>)
old_value	string (nullable)	Previous value
new_value	string (nullable)	New value
changed_by	string	Who or what triggered it (<code>system</code> , agent name)
timestamp	datetime	When

Routing Rules (Seed Data)

The system should come pre-loaded with these routing rules, evaluated in priority order (first match wins):

#	Condition	Target Team	Auto-Priority	Notes
1	category = <code>security</code>	Security	P1	All security tickets are critical
2	category = <code>billing</code> AND priority in (<code>P1</code> , <code>P2</code>)	Billing	—	High-priority billing to Billing team
3	category = <code>billing</code> AND priority in (<code>P3</code> , <code>P4</code>)	Account Management	—	Low-priority billing to AM
4	category = <code>engineering</code>	Engineering	—	All engineering tickets to Engineering
5	category = <code>account_management</code>	Account Management	—	Direct AM tickets
6	(no match — default)	Account Management	P3	Catch-all for unclassifiable tickets

SLA Deadlines

When a ticket is created, set `sla_deadline` based on priority:

Priority	Must be assigned within
P1	5 minutes
P2	1 hour
P3	24 hours
P4	48 hours

Seed Data

Teams

Name	Escalation Contact
Engineering	eng-lead@example.com
Billing	billing-lead@example.com
Account Management	am-lead@example.com
Security	security-lead@example.com

Users

Email	Role	Password
agent@example.com	Support Agent	testpass123
lead@example.com	Team Lead	testpass123

Sample Tickets

Include at least 10 tickets across all statuses and categories. At least:

- 2 P1 tickets (one assigned, one breaching SLA)
- 3 tickets in different statuses (open, in_progress, resolved, closed)
- 2 tickets routed to different teams
- 1 ticket with no matching rule (hit the catch-all default)

Requirements

Part 1 — Backend API

Build a RESTful API with the following:

Authentication

- Simple JWT-based auth with role-based access (Agent vs Team Lead)
- `/api/auth/login` and `/api/auth/me`

Ticket CRUD

- `POST /api/tickets` — Create a ticket. On creation, the system must:
 1. Evaluate routing rules in priority order (first match wins)
 2. Assign `target_team` and override `priority` if the rule specifies `auto_priority`
 3. Calculate and set `sla_deadline` based on final priority
 4. Log all actions to Ticket History
- `GET /api/tickets` — List tickets with filtering (`status` , `priority` , `category` , `assigned_team`) and pagination
- `GET /api/tickets/{id}` — Ticket detail with full history
- `PUT /api/tickets/{id}` — Update ticket (change status, assign agent, reassign team)
- Status transitions must be enforced: `open` → `in_progress` → `resolved` → `closed` .
No skipping steps.

SLA

- `GET /api/tickets/sla-breaches` — Return all tickets where `sla_deadline` has passed and `assigned_agent` is still null (unassigned)
- Include `sla_status` in ticket responses: `on_track` , `at_risk` (within 20% of deadline), `breached`

Metrics

- `GET /api/metrics` — Return:
 - Ticket count by status
 - Ticket count by priority
 - Ticket count by team
 - Number of SLA breaches (current)
 - Average resolution time (for resolved/closed tickets)

Health

- `GET /api/health` — `{"status": "healthy", "db": "connected", "timestamp": "..."}"`

Part 2 — Frontend

Build a simple, functional UI with these views:

Ticket List

- Table showing: Title, Priority, Category, Team, Status, Created, SLA Status
- Priority badges with color coding (P1 = red, P2 = orange, P3 = yellow, P4 = gray)
- SLA status indicator (green = on track, yellow = at risk, red = breached)
- Filter by status, priority, team
- Click row to open detail view

Ticket Detail

- Full ticket info + assigned team + agent
- Action buttons based on status:
 - Open: "Start Working" (→ in_progress), "Assign to me"
 - In Progress: "Resolve" (→ resolved)
 - Resolved: "Close" (→ closed), "Reopen" (→ in_progress)
- History timeline showing all actions with timestamps

Submit Ticket Form

- Fields: title, description, channel (dropdown), category (dropdown), priority (dropdown)
- On submit, show the routing result (which team it was assigned to, final priority, SLA deadline)

Metrics Dashboard (Team Lead only)

- Summary cards: total open, total in progress, SLA breaches, avg resolution time
- Ticket counts by team (table or simple bar chart)
- List of current SLA breaches with time elapsed

Part 3 — Containerization

Provide a `docker-compose.yml` that runs the full stack:

```
docker-compose up --build
```

This must include:

- **Backend** container (FastAPI, Django, Express, or equivalent)

- **Frontend** container
- **PostgreSQL** container with seed data (teams, users, routing rules, sample tickets)

Include:

- Dockerfiles for backend and frontend
- Environment variable configuration (no hardcoded secrets)
- Health check endpoint
- A seed script that populates all seed data on first run

Evaluation Criteria

1. API Design & Correctness (30 points)

- RESTful conventions (proper HTTP methods, status codes, error responses)
- Routing rules engine works correctly (priority ordering, first-match-wins, auto-priority override)
- Status transitions enforced (no invalid state changes)
- SLA calculation and breach detection is accurate
- Ticket history captures all state changes automatically
- Filtering and pagination work

2. Frontend Quality (25 points)

- Clean, functional UI (doesn't need to be beautiful, but must be usable)
- Ticket list with working filters and SLA indicators
- Detail view with correct action buttons per status
- Submit form that shows routing result
- Metrics dashboard renders with real data
- Role-based visibility (Team Lead sees metrics, Agent doesn't)

3. Routing & SLA Logic (20 points)

- Rules evaluated in correct priority order
- First-match-wins logic works (not all-match)
- Default catch-all for unmatched tickets
- Auto-priority override applied correctly
- SLA deadlines calculated correctly based on final (possibly overridden) priority
- SLA status (`on_track` / `at_risk` / `breached`) calculated correctly in real-time

4. Containerization (15 points)

- `docker-compose up --build` works on a fresh machine
- All services start and connect
- Seed data loads automatically
- Environment variables used (no hardcoded connection strings)
- Health check works

5. Code Quality (10 points)

- Clear separation of concerns
 - Consistent style and naming
 - Proper error handling
 - No security anti-patterns (SQL injection, plaintext passwords, missing auth)
-

What We're NOT Looking For

- Over-engineered solutions (microservices, Kafka, Kubernetes — this is a simple app)
- Pixel-perfect design (functional and clean is sufficient)
- 100% test coverage (but key paths like routing logic should be tested)
- Complex authentication (simple JWT is fine, no OAuth/SSO needed)

What We ARE Looking For

- Does the routing engine work correctly end-to-end?
 - Are status transitions enforced, not just suggested?
 - Is SLA tracking accurate and visible?
 - Does the containerized setup actually work out of the box?
 - Is the code organized in a way a team could understand and extend?
-

Deliverables

1. **Working code** — GitHub repo (public or invite us)
 2. **README** — Setup instructions (`docker-compose up` should be the main instruction), plus a short section on architecture decisions and tradeoffs
 3. **Short writeup (half page)** — Answer these two questions:
 - How would you evolve this system if ticket volume went from 500/day to 50,000/hour? What architectural changes would you make?
 - If the database went down for 30 seconds while tickets are still coming in, what happens in your current design? What would you change?
-

Submission

Submit a link to your GitHub repo within 24 hours of receiving this assessment.

Note: This assessment will be discussed in a follow-up interview. Be prepared to walk through your routing logic, explain your architecture decisions, and discuss how your design would handle scale and failure scenarios.